Nebula: Decoupling Agent Identity from Runtime in Multi-Agent LLM Systems

Tianmou Zhang
Enigma Entertainment
Hong Kong
jaydenz@enigmaetmt.com

***Abstract*—As LLM-based agents move from single-session tools to persistent, collaborative systems, a fundamental architectural question arises: who owns the agent's state? In most existing frameworks, the execution runtime (a CLI process, an API client, or an in-memory object) owns the conversation history, and the orchestration layer has limited visibility into or control over individual messages. This paper describes Nebula, an open-source multi-agent platform that inverts this relationship by storing all agent state—messages, skills, knowledge, credentials, and memory—in a normalized relational database, separate from any execution runtime. We formalize this as a *soul/body separation*, where the soul (persistent identity) is composed into context-appropriate prompts and injected into disposable runtime processes (bodies) at each execution. We describe the architectural patterns this separation enables and discuss the trade-offs involved.**

***Index Terms*—LLM agents, multi-agent systems, agent orchestration, soul/body separation, context composition, persistent memory, runtime abstraction**

I. Introduction

A. The Problem

The current generation of LLM coding agents—CLI tools like Claude Code [1] and Aider [2], IDE-integrated agents like Cursor [3], and sandboxed platforms like OpenHands [4]—are effective tools for individual tasks. A user interacts with one agent in one session, and the agent maintains its own conversation state internally. While some tools support subagent spawning within a session (e.g., Claude Code's Agent tool, OpenHands' agent delegation), the orchestration is scoped to a single execution—there is no persistent multi-agent identity or cross-session coordination. This model breaks down when we want:

- **Multiple agents** collaborating on different aspects of the same project
- **Persistent agents** that survive infrastructure failures (container restarts, session eviction)
- **Context-dependent behavior** where the same agent operates differently depending on what it is working on
- **Inter-agent communication** where agents can request help from peers without pre-configured workflows

These requirements arise naturally in organizational settings where AI agents take on specialized roles and need to coordinate.

B. Observation

In all of the above scenarios, the underlying issue is the same: **the execution runtime owns the agent's state, and the orchestration layer cannot inspect, compose, or route it**. If the runtime's session file is lost, the conversation is gone. If two agents need to exchange context, there is no shared representation to draw from. If an agent needs different capabilities in different projects, there is no mechanism to swap them without reconfiguring the runtime.

C. Approach

Nebula separates agent *identity* from agent *execution*:

- The **soul** is the agent's persistent identity—its name, role, accumulated knowledge, skills, credentials, memory, and complete message history—stored in a relational database.
- The **body** is a disposable CLI process that receives a context-appropriate subset of the soul at spawn time, executes a task, and returns.

This is not a novel concept in distributed systems—the separation of state from compute is well-established. Our contribution is applying it to LLM agent orchestration and documenting the architectural patterns it enables and the trade-offs it introduces. Figure 1 illustrates the overall structure.

II. Related Work

In-memory agent frameworks. LangChain [5], AutoGen [6], and CrewAI [7] provide multi-agent abstractions in Python. LangGraph (LangChain's agent runtime) introduced durable checkpointing that can survive process restarts, and CrewAI added opt-in state persistence. However, inter-agent communication still requires explicit wiring (group chats, pub-sub buses, delegation actions), and agent identity—the composition of skills, secrets, memory, and knowledge—remains coupled to the orchestrating process rather than stored as an independently addressable entity. MetaGPT [8] introduces role-based

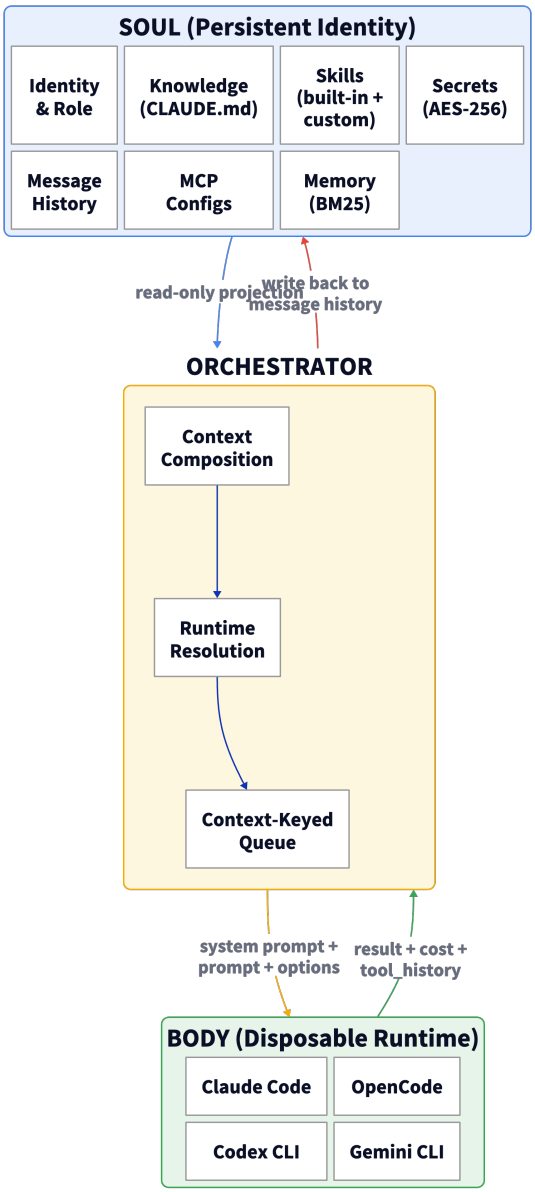

Fig. 1. Soul/body separation. The soul (persistent agent identity) is stored in a relational database and composed into a context-appropriate system prompt at each execution. The body is a disposable CLI process that receives this projection, executes the task, and returns structured output.

agents with a shared message pool but does not separate agent identity from runtime state.

Agent orchestration platforms. OpenHands [4] provides sandboxed environments for coding agents with support for agent delegation (e.g., CodeActAgent delegating to BrowsingAgent), but the multi-agent coordination occurs within a single execution context—agents do not persist as independently owned identities across sessions. Workflow engines (Windmill, n8n, Temporal) orchestrate tasks via pre-defined DAGs, which are effective for known workflows but cannot adapt to the ad-hoc coordination patterns that emerge in agent collaboration.

CLI coding agents. Claude Code [1], Aider [2], OpenAI Codex CLI [11], and Google Gemini CLI [12] manage their own session state (conversation files, session IDs). Some support subagent spawning within a session (Claude Code's Agent tool) or programmatic orchestration (Claude Code's Agent SDK), but each tool's state management, skill system, and session persistence remain self-contained. Nebula treats these CLIs as interchangeable execution runtimes—the agent's identity, memory, skills, and credentials are managed externally and injected at each execution, independent of which CLI is used.

Actor model and agent communication. The Actor model [9] and FIPA-ACL [10] provide theoretical foundations for message-passing concurrency. Nebula's @mention routing shares conceptual lineage but operates at the natural language level—messages are human-readable text, not structured protocols—prioritizing inspectability and debuggability over formal verification.

III. Architecture

A. Overview

Nebula is a single-container application (Node.js 22, Express, React, SQLite via better-sqlite3) that manages multiple agents within user-scoped organizations. The core data model:

```
User (email/password or OAuth)
 +-- Organization
      +-- Agents
      |    +-- Conversations -> Messages
      |    +-- Tasks (cron / webhook)
      |    +-- Custom Skills
      |    +-- Secrets (AES-256-GCM)
      |    +-- MCP Server Configs
      |    +-- Memories (BM25-indexed)
      +-- Projects
      |    +-- Milestones -> Deliverables
      |    +-- Project Agents (coord/contrib)
      |    +-- Project Secrets
      |    +-- Project Memories (shared)
      +-- Custom Skills (org-wide)
      +-- MCP Servers (org-wide)
      +-- Secrets Vault (org-wide)
```

Each message is stored as an individually addressable database row with its own ID, role, content, type classification, and metadata (execution duration, cost, tool history). This granularity—rather than storing conversations as opaque blobs—is what enables the routing, recovery, and composition patterns described below.

B. Soul: Persistent Agent Identity

An agent's soul comprises seven components, all stored outside any runtime process:

1) **Identity & Role**—name, role description, model selection, tool permissions

2) **Knowledge**—org-wide and project-specific CLAUDE.md files
3) **Skills**—built-in (14 system skills, context-dependent) + custom (user-defined, org or agent scoped)
4) **Secrets**—AES-256-GCM encrypted, three scopes (org, agent, project) with context-dependent resolution
5) **Message History**—every message individually addressable by ID
6) **MCP Configs**—Model Context Protocol server configurations (org + agent scoped)
7) **Memory**—persistent knowledge entries with BM25 full-text search index

The soul is never modified by the runtime directly. The runtime receives a read-only projection of the soul (the composed system prompt) and returns structured output (result text, cost, tool invocations) that the orchestrator writes back to the soul's message history.

C. Body: Disposable Runtime Execution

A body is a short-lived process that implements a common execution interface:

```
execute({ prompt, systemPrompt, agentDir,
          conversation, options })
  -> { result, duration_ms, total_cost_usd,
       usage, tool_history }
```

The orchestrator supports four CLI runtimes via a pluggable adapter registry (`src/backends/cli-registry.js`):

TABLE I
SUPPORTED RUNTIME BACKENDS

Runtime	Binary	Resume	Skills	MCP
Claude Code	`claude`	`--resume`	Disk	File
OpenCode	`opencode`	`--session`	Inline	JSON
Codex CLI	`codex`	`resume`	Inline	—
Gemini CLI	`gemini`	`--resume`	Inline	—

Adding a new runtime requires one adapter file implementing a base class (`src/backends/base.js`) and one line of registration. The adapter specifies how to spawn the process, how to inject skills (disk vs. system prompt), how to pass MCP configs, and how to parse output for session IDs and cost metadata.

The registry resolves which runtime to use per execution: agent's configured backend → org default → any available CLI that can run the requested model → error.

Additionally, a WebSocket bridge enables remote execution on external machines via a Tauri desktop app or headless CLI client.

IV. ARCHITECTURAL PATTERNS

A. Dynamic Context Composition

At each execution, the orchestrator assembles a system prompt from the soul's components, filtered by execution context. The composition pipeline in `src/services/executor.js` proceeds in this order:

1) Global knowledge (org-wide CLAUDE.md)
2) Project knowledge (branch-specific CLAUDE.md, if in project context)
3) Built-in skills (context-dependent subset of 14 system skills, including nebula-tasks, nebula-mail, nebula-agents, nebula-memory, nebula-projects, nebula-nas, nebula-coding-conventions)
4) Integration skills (dynamically generated from project links: Gitea, GitHub, GitLab, YouTrack, etc.)
5) Project skills (coordinator or contributor skill set, if in project context)
6) Custom skills (org-wide + agent-specific, with secret template interpolation)
7) Memory index (titles and descriptions of all agent memories + project memories)
8) Peer agent directory (names, roles, and availability of sibling agents)
9) Agent identity (role, working directory, security guidelines)

The composition is **not cached**—it is regenerated per execution. A skill update, secret rotation, memory mutation, or project reassignment takes effect on the next execution without any restart or cache invalidation.

Secret interpolation across trust boundaries. Skills reference credentials using `{{KEY}}` template syntax. The resolver applies different strategies depending on the consumer: agents see environment variable references (`${K}`), never plaintext values. MCP server processes—which are trusted executables, not LLM-controlled—receive the actual decrypted credentials. This creates a security boundary: agents can *invoke* credentialed tools but cannot *read* the credentials themselves.

B. Persistent Memory with BM25 Search

Agents accumulate persistent knowledge via an API-managed memory system stored in the `memories` table. Each memory has a title, description, type (user, feedback, project, reference), and content body. An in-memory BM25 index (parameters: K1=1.2, B=0.75) provides full-text search with title-boosted ranking (title weighted 3×).

Memory access is context-dependent:

TABLE II
MEMORY ACCESS BY EXECUTION CONTEXT

Context	Writes to	Reads from
Main conv.	Agent mem.	Agent mem.
Project conv.	Project mem.	Agent (RO) + Project
Task stack	Rejected	Agent (RO) + Project (RO)

Progressive disclosure. Memory titles and descriptions (∼60 characters per concept) are injected into the

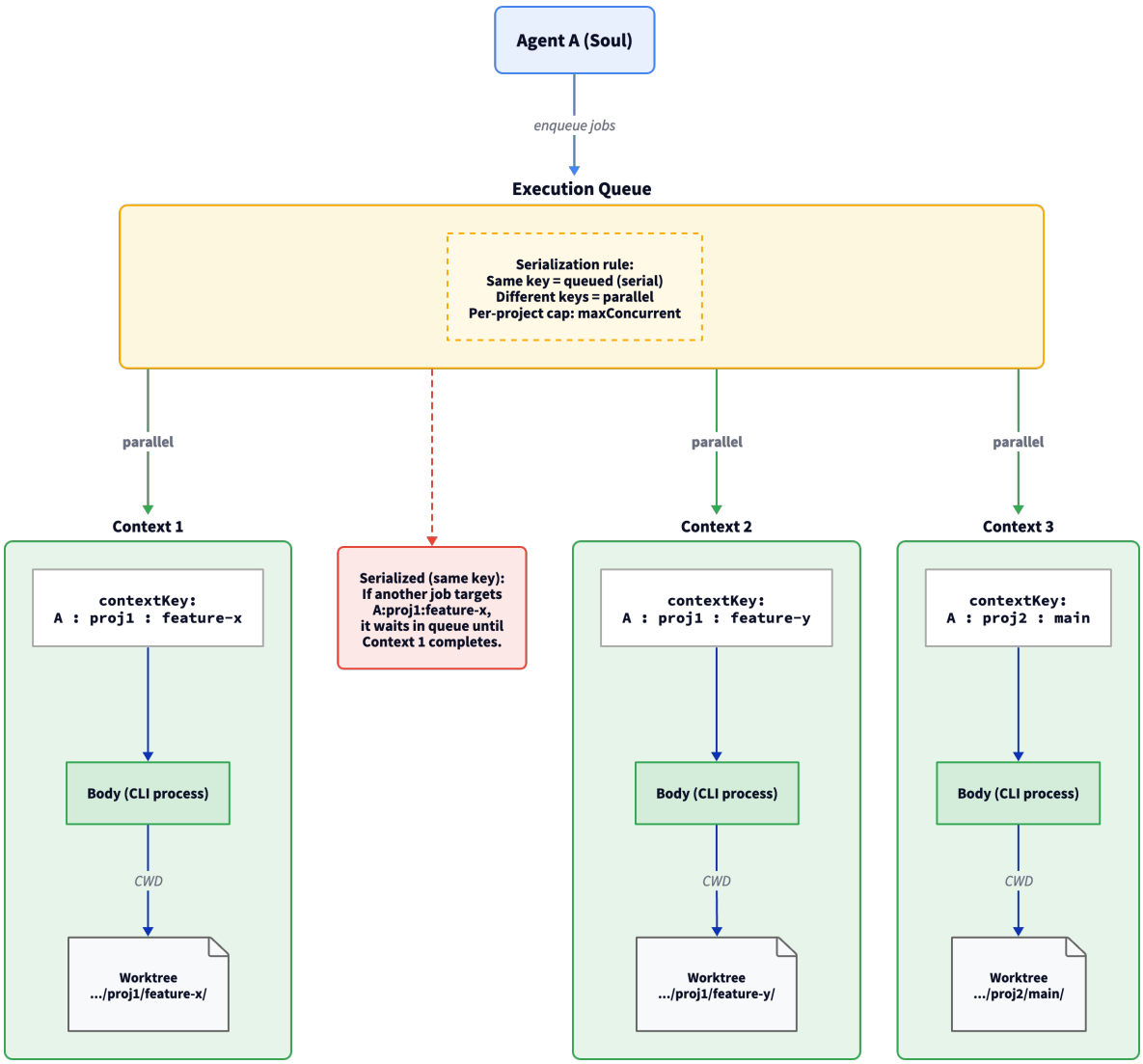

Fig. 2. Context-keyed concurrent execution. One agent spawns three independent bodies in parallel, each bound to a distinct `(agentId, projectId, branch)` key and isolated worktree. Jobs sharing a key are serialized; jobs with different keys run concurrently.

system prompt at every execution, giving the agent awareness of what it knows without consuming full content tokens. The agent searches and reads specific memories on demand via HTTP endpoints exposed through the `nebula-memory` built-in skill.

External knowledge base fan-out. Memory search optionally fans out to external KBs linked to the current project—Confluence, Notion, YouTrack—via adapter functions in `src/services/kb-search.js`. External queries timeout at 5 seconds; local BM25 results are always returned regardless of external failures.

The BM25 index is rebuilt in-memory on every memory mutation and on server start. At typical corpus sizes (30–120 documents per scope), rebuild time is negligible.

C. Context-Keyed Concurrent Execution

A single agent can execute simultaneously across multiple contexts. The execution queue is partitioned by a three-dimensional key:

```
contextKey(agentId, projectId, branchName)
    = agentId:projectId:branchName
```

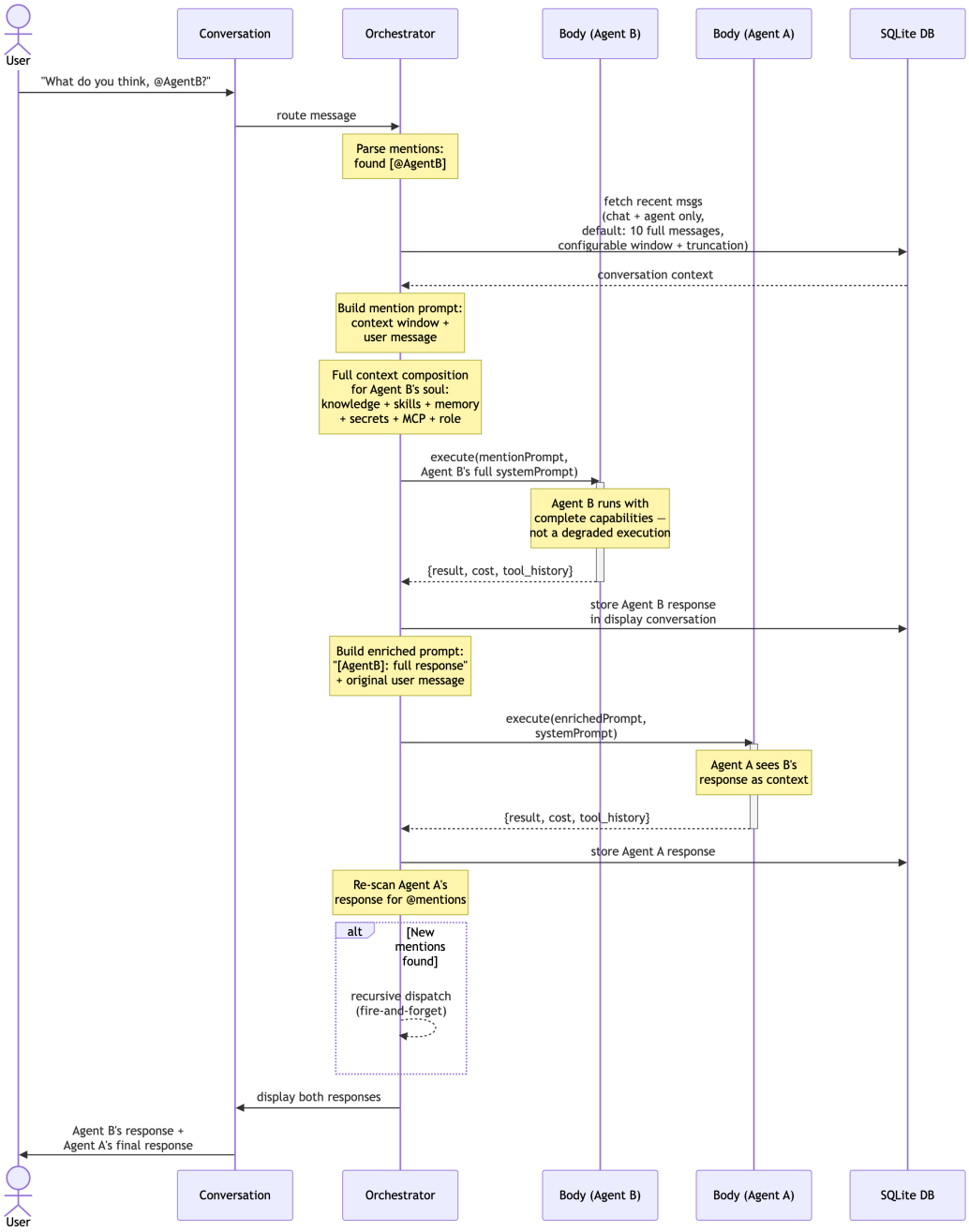

Fig. 3. Synchronous @mention routing sequence. The orchestrator parses the mention, builds a context window, dispatches the mentioned agent with its full soul composed, and injects the full response back before executing the originating agent.

Serialization rule: Jobs with the same context key are serialized (queued). Jobs with different keys execute in parallel, subject to a per-agent concurrency cap per project (`maxConcurrent` in `project_agents`).

Isolation mechanism: Each context maps to a distinct git worktree (path: `.../agents/{agentId}/projects/{projectId}/{branch}/`), providing filesystem-level isolation between concurrent bodies. No shared mutable state exists between concurrent executions. Figure 2 illustrates this fan-out.

D. *Message-Driven Inter-Agent Routing*

Inter-agent communication uses two natural-language routing primitives embedded in message content.

@mention (synchronous). When a message contains `@AgentName`, the orchestrator: (1) parses all mentions (up to 3 per message); (2) builds a configurable context window from the originating conversation (default: last 10 messages, untruncated; both window size and per-message truncation are tunable); (3) executes each mentioned agent in its own session with the context injected and the agent's

full soul composed into its system prompt; (4) passes all full responses back to the originating agent as synthesized context; (5) executes the originating agent, which produces the final response.

@notify (asynchronous). `@notify AgentName` pushes a notification to the target agent's own conversation with lower queue priority. The notification includes the source agent, source conversation context, and the triggering message. The sender does not wait for the result.

Recursive routing. After any agent execution, the orchestrator re-scans the response text for `@mention` patterns and dispatches recursively. This enables multi-hop coordination workflows without explicit configuration—the topology emerges from agents' natural language decisions about whom to reference. Figure 3 shows the full synchronous dispatch sequence for a single `@mention`.

E. Session Continuity

Because the orchestrator owns all messages, session recovery does not depend on the runtime's internal state. When a CLI session is lost (container restart, eviction, branch change), the orchestrator: (1) detects the failure (runtime reports "session not found" or returns a non-resumable error); (2) queries the conversation's message history from the database; (3) constructs a recovery preamble from recent messages, filtered for noise, within a configurable token budget (default: 25,000 tokens, approximated at ∼4 characters per token); (4) resets the session ID and retries execution with the preamble prepended to the new prompt.

The recovery budget is a tunable parameter, not a hard limitation—the database contains the complete history. Full-history recovery is architecturally possible but trades token cost for context completeness.

Branch change handling. When an agent's assigned deliverable changes branches, the orchestrator detects the mismatch between the conversation's `session_branch` and the new execution context, resets the session, and injects recovery context automatically.

F. Scoped Secret Resolution

Credentials are stored at three scopes with context-dependent resolution:

- **Org secrets**—base layer, available to all agents in the org
- **Agent secrets**—override org secrets of the same key when executing in agent context
- **Project secrets**—override org secrets of the same key when executing in project context

Agent and project secrets are **sibling scopes**, not a hierarchy. An agent executing in project context receives org + project secrets; in agent context, org + agent secrets. This prevents credential leakage across project boundaries—the same agent assigned to two projects receives different credentials depending on which project it is currently executing in.

All secrets are encrypted at rest with AES-256-GCM (per-value IV and auth tag). The `SecretsList` UI component enforces write-only semantics—values can be set and deleted but never displayed after creation.

G. Remote Agent Bridging

Agents can execute on external machines via a WebSocket bridge. Two client implementations exist: a Tauri desktop app (macOS, Windows) and a headless Node.js CLI client.

The protocol transfers the complete execution context per request: prompt, system prompt, session state, allowed tools, model, runtime selection, timeout, skills (as JSON), MCP server configs, and images (as base64). The system prompt and skills are regenerated and sent fresh with every request—no caching, no synchronization protocol, no stale state.

This simplifies the protocol at the cost of bandwidth (system prompts range 20–100 KB), a trade-off that is acceptable for the execution frequencies observed in practice.

V. Multi-Agent Coordination via Projects

A. Role-Based Capability Injection

When agents are assigned to a project, each receives a role (coordinator or contributor) that determines which skills are injected at execution time:

TABLE III
Coordinator vs. Contributor Role Capabilities

Aspect	Coordinator	Contributor
Visibility	All deliverables and members	Own deliverables only
Skills	Mgmt, readiness, PR review, milestones	Branch work, updates, PR creation
Autonomy	Autonomous; stop for blockers	Follow assignment; submit for review

The same agent can be coordinator on one project and contributor on another, receiving different capabilities depending on context. This is a consequence of the soul/body separation: the soul defines *who the agent is*, but the orchestrator determines *what it can do* at each execution.

B. Concurrent Branch Development

Each deliverable maps to a git branch and a worktree, yielding per-branch isolation with no shared mutable state between concurrent executions. The same agent working on three branches spawns three independent bodies, each in its own worktree, with its own CLI session.

Knowledge evolution is handled by git itself: each branch's CLAUDE.md is a versioned file. When branches merge, knowledge merges. Conflicts are resolved by the coordinator as part of PR review—no custom conflict resolution protocol is needed.

C. Readiness as a Live Invariant

Projects implement readiness as a function evaluated on demand, not a static flag:

```
ready(project) =
    all(system_checks_met) AND all(agent_checks_met)
```

System checks are mechanically derived from project state: git remote configured, design/tech specs exist in vault, milestones have deliverables assigned, webhook configured. Agent checks are user-declared prerequisites stored in the `project_checklist` table.

Auto-demotion. If `ready(p)` evaluates to false while the project status is `active`, the system automatically demotes to `not_ready`. This is implemented in `src/services/readiness.js` and runs on every readiness evaluation.

One-way promotion. Promotion from `not_ready` to `active` requires explicit user action via `POST /api/projects/:id/launch`—it never happens automatically. This is a safety valve: the system can detect problems and halt, but only humans can authorize forward progress.

VI. Discussion

A. What the Separation Enables

The soul/body separation is not interesting in itself—it is a well-known pattern (stateless compute, externalized state). What is worth examining is the specific capabilities it enables in the LLM agent context:

Granular message routing. Because every message is an addressable database row (not buried in a session file), the orchestrator can build arbitrary context windows—last N messages from conversation A, inject into agent B's execution, store B's response in conversation A. This is the mechanism behind @mention routing, and it requires no cooperation from the runtime.

Non-destructive context switching. An agent can work on project A, switch to project B, and return to project A without losing context in either. The orchestrator maintains separate conversations (with separate session IDs and branch tracking) per context, and the runtime does not need to know about the others.

Runtime portability. When the soul is external, switching from one CLI tool to another is a configuration change, not a migration. The agent's message history, skills, memory, and knowledge persist regardless of which runtime executes the next prompt. Claude Code, OpenCode, Codex CLI, and Gemini CLI each have different strengths, and the choice can be made per-agent or per-execution.

Persistent memory across runtimes. Because memory is stored in the database (not in runtime-specific files), an agent's accumulated knowledge survives runtime changes, container restarts, and session resets. The BM25 index is rebuilt from the database on startup—memory is never lost.

B. Trade-offs

Prompt size. Dynamic context composition produces system prompts that range from 20 KB to 100 KB, depending on the number of skills, peer agents, memory entries, and project context. This is workable with current large-context models but may become a concern as the number of skills grows. Prompt compression or skill summarization could mitigate this.

Single-instance storage. The current SQLite implementation is appropriate for single-instance deployment but does not scale horizontally. The architectural principles—soul/body separation, context-keyed queuing, scoped secrets—are not inherently tied to SQLite and could be implemented on a distributed store.

Context key correctness. The per-context serialization model depends on correct construction of the `contextKey(a,p,b)` triplet. An error in key generation could allow concurrent execution within the same worktree. The current implementation has not exhibited this failure, but it is not formally verified.

Recovery fidelity vs. cost. Session recovery injects historical messages into the new prompt, consuming tokens. The default budget (25,000 tokens, ∼100,000 characters) was chosen empirically. Full-history recovery is possible but expensive for long conversations. The right budget depends on the use case.

BM25 vs. vector search. The memory system uses BM25 (term-frequency based) rather than vector embeddings for search. At typical per-agent corpus sizes (30–120 memories), BM25 with title boosting performs well for keyword-oriented queries without requiring an embedding model or external service. This is a deliberate simplicity trade-off—vector search would add latency and infrastructure cost for marginal accuracy gains at this scale.

VII. Conclusion

This paper describes an architectural pattern—separating agent identity from execution runtime—and documents the capabilities it enables for multi-agent LLM systems. The core mechanism is straightforward: store agent state in a database, compose context-appropriate prompts at each execution, and treat runtime processes as disposable.

The resulting system supports concurrent multi-context execution, transparent session recovery, message-driven inter-agent routing, persistent BM25-indexed memory, runtime-agnostic deployment, and role-based project coordination—capabilities that are difficult to achieve when the runtime owns the agent's state.

Nebula is released under AGPL-3.0 at https://github.com/reforia/Nebula.

Acknowledgment

Substantial portions of the Nebula codebase were developed with the assistance of Claude Code (Anthropic). This paper was drafted and revised with AI assistance.

References

[1] Anthropic, "Claude Code," https://docs.anthropic.com/en/docs/claude-code, 2024.

[2] P. Gauthier, "Aider: AI pair programming in your terminal," https://github.com/paul-gauthier/aider, 2023.

[3] Cursor, https://cursor.com, 2023.

[4] X. Wang et al., "OpenHands: An Open Platform for AI Software Developers as Generalist Agents," arXiv:2407.16741, 2024.

[5] H. Chase, "LangChain," https://github.com/langchain-ai/langchain, 2022.

[6] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023.

[7] J. Moura, "CrewAI," https://github.com/joaomdmoura/crewAI, 2024.

[8] S. Hong et al., "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," arXiv:2308.00352, 2023.

[9] C. Hewitt, P. Bishop, R. Steiger, "A Universal Modular ACTOR Formalism for Artificial Intelligence," *IJCAI*, 1973.

[10] FIPA, "FIPA ACL Message Structure Specification," Foundation for Intelligent Physical Agents, 2002.

[11] OpenAI, "Codex CLI," https://github.com/openai/codex, 2025.

[12] Google, "Gemini CLI," https://github.com/google-gemini/gemini-cli, 2025.